

Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

## **ОТЧЕТ**

по лабораторной работе №5

по дисциплине «Разработка кроссплатформенных приложений»

на тему «Многопоточность в Java»

Вариант №10

Выполнили: ст. гр. 23ВВИ1

Галкин А.П.

Казаков А.Д.

Приняли:

к.т.н. доцент Юрова О.В.

к.т.н. доцент Карамышева Н.С.

Пенза 2026

**Цель работы:** научиться создавать многопоточные приложения с использованием стандартных средств языка Java.

### **Задание:**

Модифицировать приложение из предыдущей лабораторной работы, реализовав вычисление определенного интеграла в нескольких дополнительных потоках (число потоков определяется номером варианта), снимая нагрузку с основного потока и предотвращая "подвисание" графического интерфейса. Варианты с номерами до 5 включительно реализуют многопоточность путем наследования от класса Thread, остальные реализуют интерфейс Runnable. Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

### **Ход работы:**

1. Так как вариант №10 > 5, многопоточность реализована через интерфейс Runnable.
2. Создан пул потоков ExecutorService с фиксированным размером 10 (по номеру варианта).
3. Разработан класс CalculationTask, реализующий Runnable, который принимает параметры строки таблицы и запускает вычисление в фоновом потоке.
4. Для ускорения вычислений интеграл разбивается на 10 равных частей. Каждая часть вычисляется в отдельной задаче PartialIntegralTask, реализующей интерфейс Callable<Double>.
5. Метод calculateParallel собирает результаты всех частей через Future, суммирует их и возвращает итоговое значение.
6. Обновление ячейки таблицы выполняется строго в потоке EDT через SwingUtilities.invokeLater(), что предотвращает конфликт потоков и "подвисание" интерфейса.
7. При закрытии окна приложения пул потоков корректно завершается методом executor.shutdown().

### **Листинг программы:**

Инициализация пула потоков (JForm.java):

```
private ExecutorService executor = Executors.newFixedThreadPool(10);
```

Класс задачи вычисления части интеграла (JForm.java):

```
class PartialIntegralTask implements Callable<Double> {  
    private double a, b, step;  
  
    public PartialIntegralTask(double a, double b, double step) {  
        this.a = a; this.b = b; this.step = step;  
    }  
  
    @Override  
    public Double call() throws Exception {  
        String threadName = Thread.currentThread().getName();  
  
        double sum = 0;  
  
        for (double x = a; x < b; x += step) {  
            double h = step;  
  
            if (x + step > b) h = b - x;  
  
            double fx = Math.exp(x) / x;  
  
            sum += fx * h;  
        }  
  
        System.out.println("Поток " + threadName + " вычислил часть интеграла от " + a + " до "  
+ b + " = " + sum);  
  
        return sum;  
    }  
}
```

Метод параллельного вычисления (JForm.java):

```
private double calculateParallel(double low, double up, double step) throws Exception {  
    double intervalLength = up - low;  
  
    double partLength = intervalLength / 10;
```

```

List<Callable<Double>> tasks = new ArrayList<>();
for (int i = 0; i < 10; i++) {
    double partA = low + i * partLength;
    double partB = (i == 9) ? up : low + (i + 1) * partLength;
    tasks.add(new PartialIntegralTask(partA, partB, step));
}
List<Future<Double>> futures = executor.invokeAll(tasks);
double totalSum = 0;
for (Future<Double> future : futures) totalSum += future.get();
return totalSum;
}

```

Класс фоновой задачи (CalculationTask.java):

```

package pkg23vvv1;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;

class CalculationTask implements Runnable {
    private RecIntegral recIntegral;
    private DefaultTableModel tableModel;
    private int selectedRow;

    public CalculationTask(RecIntegral recIntegral, DefaultTableModel tableModel, int
selectedRow) {
        this.recIntegral = recIntegral;
        this.tableModel = tableModel;
        this.selectedRow = selectedRow;
    }
}

```

```

@Override

public void run() {

    String threadName = Thread.currentThread().getName();

    System.out.println("Главный поток " + threadName + " начинает вычисление интеграла
для строки " + selectedRow);

    try {

        double result = calculateParallel(recIntegral.GetLowLim(), recIntegral.GetUpperLim(),
recIntegral.GetStep());

        recIntegral.setResult(result);

        SwingUtilities.invokeLater(() -> {

            tableModel.setValueAt(String.format("%.10e", recIntegral.GetResult()), selectedRow,
3);

        });

    } catch (Exception e) {

        System.err.println("Ошибка: " + e.getMessage());

        SwingUtilities.invokeLater(() -> {

            tableModel.setValueAt("Ошибка: " + e.getMessage(), selectedRow, 3);

            JOptionPane.showMessageDialog(null, "Ошибка вычисления: " + e.getMessage());

        });

    }

}

}

```

Обработчик кнопки "Вычислить" (JForm.java):

```

private void jButtonSolveActionPerformed(java.awt.event.ActionEvent evt) {

    int selectedRow = jTable1.getSelectedRow();

    if (selectedRow != -1) {

        try {

```

```

double a = Double.parseDouble(jTable1.getValueAt(selectedRow, 0).toString());
double b = Double.parseDouble(jTable1.getValueAt(selectedRow, 1).toString());
double step = Double.parseDouble(jTable1.getValueAt(selectedRow, 2).toString());
RecIntegral ri = new RecIntegral(a, b, step);

CalculationTask task = new CalculationTask(ri, (DefaultTableModel)jTable1.getModel(),
selectedRow);

executor.submit(task);

tableContent.add(ri);

javax.swing.SwingUtilities.invokeLater(() -> {

    jTable1.setValueAt("Вычисляется...", selectedRow, 3);

});

} catch (NumberFormatException ex) {

    JOptionPane.showMessageDialog(this, "Введите корректные числа (разделитель
дробной части - точка). ", "Ошибка ввода", JOptionPane.ERROR_MESSAGE);

} catch (Exception e) {

    JOptionPane.showMessageDialog(this, e.getMessage(), "Ошибка",
JOptionPane.ERROR_MESSAGE);

}

}

}

```

**Скриншоты:**

Меню

Формула:  $(e^x)/x$

Нижний порог

Верхний порог

Шаг

Добавление из коллекции

Очистить таблицу

Добавить

Удалить

Вычислить

| Нижний порог | Верхний порог | Шаг   | Результат |
|--------------|---------------|-------|-----------|
| 1            | 50            | 0.01  |           |
| 2            | 100           | 0.005 |           |
| 0.5          | 75            | 0.01  |           |

Рисунок 1 – Интерфейс перед запуском вычислений

Меню

Формула:  $(e^x)/x$

Нижний порог

Верхний порог

Шаг

Добавление из коллекции

Очистить таблицу

Добавить

Удалить

Вычислить

| Нижний порог | Верхний порог | Шаг   | Результат        |
|--------------|---------------|-------|------------------|
| 1            | 50            | 0.01  | 1.0533874525e+20 |
| 2            | 100           | 0.005 | 2.7088379962e+41 |
| 0.5          | 75            | 0.01  |                  |

Рисунок 2 – Процесс вычисления в фоновых потоках (некоторое время в колонке Результат пишется Вычисление)

```

Главный поток pool-1-thread-1 начинает вычисление интеграла для строки 2
Поток pool-1-thread-10 вычислил часть интеграла от 7.95 до 15.4 = 338643.3914467314
Поток pool-1-thread-2 вычислил часть интеграла от 15.4 до 22.85 = 3.825902284235417E8
Поток pool-1-thread-4 вычислил часть интеграла от 22.85 до 30.3 = 4.90189155790626E11
Поток pool-1-thread-10 вычислил часть интеграла от 30.3 до 37.75 = 6.718460499751431E14
Поток pool-1-thread-2 вычислил часть интеграла от 37.75 до 45.2 = 9.604922212938917E17
Поток pool-1-thread-4 вычислил часть интеграла от 45.2 до 52.65 = 1.4134402620585523E21
Поток pool-1-thread-10 вычислил часть интеграла от 52.65 до 60.1 = 2.1242921159094774E24
Поток pool-1-thread-2 вычислил часть интеграла от 60.1 до 67.55 = 3.244295151038319E27
Поток pool-1-thread-4 вычислил часть интеграла от 67.55 до 75.0 = 5.0177756450013966E30
Поток pool-1-thread-7 вычислил часть интеграла от 0.5 до 7.95 = 419.931980912487

```

Рисунок 3 – Вывод в консоль о работе потоков

Меню

Формула:  $(e^x)/x$

Нижний порог:

Верхний порог:

Шаг:

Добавление из коллекции

Добавить

Очистить таблицу

Удалить

Вычислить

| Нижний порог | Верхний порог | Шаг   | Результат        |
|--------------|---------------|-------|------------------|
| 1            | 50            | 0.01  | 1.0533874525e+20 |
| 2            | 100           | 0.005 | 2.7088379962e+41 |
| 0.5          | 75            | 0.01  | 5.0210220659e+30 |

Рисунок 4 – Итоговый результат вычисления

Вывод: в ходе выполнения лабораторной работы освоены принципы многопоточного программирования в Java. Для варианта №10 реализован интерфейс Runnable, создан пул потоков ExecutorService фиксированного размера (10 потоков). Вычисление интеграла распараллелено: интервал разбивается на части, каждая обрабатывается в отдельном потоке через Callable и Future. Обновление графического интерфейса вынесено в Event Dispatch Thread с помощью SwingUtilities.invokeLater(), что исключило блокировку UI и обеспечило потокобезопасность. Получены навыки управления жизненным циклом потоков и агрегации результатов параллельных вычислений.